

[← Go Back](#)

Hardware

Portenta Machine Control ▼

Tutorials

[Memory Partitioning for Use with the Arduino IDE](#)[Arduino Portenta Machine Control Library Guide](#)[Modbus RTU On Portenta Machine Control Using PLC IDE](#)[Modbus TCP On Portenta Machine Control Using PLC IDE](#)[Modbus TCP with Portenta Machine Control & Opta™](#)[Tank Thermoregulation with Portenta Machine Control & Opta™](#)[Connect an RTD/Thermocouple to the Portenta Machine Control](#)[Portenta Machine Control User Manual](#)

[Home](#) / [Hardware](#) / [Portenta Machine Control](#) / [Modbus RTU On Portenta Machine Control Using PLC IDE](#)

Modbus RTU On Portenta Machine Control Using PLC IDE

Learn to set and enable Modbus RTU on the Portenta Machine Control using the Arduino® PLC IDE.

Taddy Chung
Author and Last revision · 10/03/2024
José Bagur

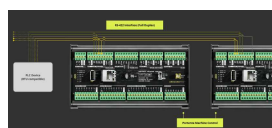
ON THIS PAGE

Overview

[Goals](#)[Required Hardware and Software](#) +[Modbus Protocol](#)[Modbus RTU & PLC IDE](#)[Instructions](#) +[Conclusion](#) +

Overview

The Portenta Machine Control boasts adaptable, high-quality industrial hardware and offers various connection options. Its capabilities are further amplified by the Arduino PLC IDE software, optimizing the device for robust field operations. Moreover, the Portenta Machine Control is compatible with Modbus protocols. With the Arduino PLC IDE, their integration is made straightforward.



Portenta Machine Control with Modbus

[Help](#)

In this tutorial, you will discover how to establish Modbus RTU communication between two Portenta Machine Control devices through the Arduino PLC IDE.

Goals

Learn to configure the workspace environment to work with Modbus RTU using the Arduino PLC IDE.

Configure Modbus RTU for Portenta Machine Control using the Arduino PLC IDE.

Learn to verify that the Portenta Machine Control has been correctly set up using a Modbus RTU communication example.

Required Hardware and Software

Hardware Requirements

Portenta Machine Control (x2)

Micro USB cable (x2)

Cable with either specification for RS-485/RS-422 connection (x3):

STP/UTP 24-18AWG (Unterminated) 100-130Ω rated

STP/UTP 22-16AWG (Terminated) 100-130Ω rated

For the RS-422 interface (Full-Duplex mode), termination resistors must be placed on the **Receiver sides** of both the Client device and the furthest Server device.



Software Requirements

Arduino PLC IDE Installer

If you have a Portenta Machine Control, your device will need a unique PLC IDE License key. You can get your license key [here](#). To learn more, go to the **License**

Activation with Product Key (Portenta Machine Control) section.

[Portenta Machine Control Modbus RTU PLC IDE Project Example File](#)

Modbus Protocol

Modbus is an open, client/server architecture-based serial communication protocol free of licensing restrictions. It is commonly employed in industrial electrical equipment, particularly in Building Management Systems (BMS) and Industrial Automation Systems (IAS).

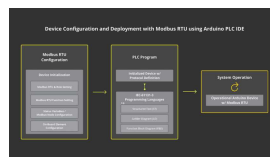
It was released by Modicon (now Schneider Electric) in 1979. Since then, it has become the de facto industry standard for industrial

In Supervisory Control and Data Acquisition (SCADA) systems, a Remote Terminal Unit (RTU) and a supervisory device are frequently connected via the Modbus communication protocol. Modbus messages have a straightforward 16-bit format and a Cyclic-Redundant Checksum (CRC) when communicating with electronic equipment.

Modbus RTU & PLC IDE

This tutorial will walk you through setting up two Portenta Machine Control devices for Modbus RTU communication over the RS-485 interface using the Arduino PLC IDE. This tutorial clarifies the steps required for successful implementation.

The accompanying diagram offers a clear representation of the Portenta Machine Control configuration and deployment process for Modbus RTU:



Modbus RTU Implementation with Arduino PLC IDE

The entire procedure is divided into three distinct stages:

Modbus RTU

Configuration serves as the initial step, wherein we configure the Portenta Machine Control device with Modbus RTU and other inherent properties.

In this phase, the role of the Portenta Machine Control device as either a Client or Server within Modbus RTU is specified. It includes settings such as *Port type*, *Baud rate*, and *Serial Mode*, which are crucial for the communication protocol.

Depending on the device's role within Modbus RTU, we either outline the 'Status variables' or define the Modbus node to identify the devices communicating under this protocol.

The **PLC Program** is created as a post-device configuration, integrating Modbus RTU and other functionalities. This stage eliminates the need for detailed Modbus RTU configurations within the PLC code.

The setup is user-friendly. The system autonomously manages Modbus RTU data exchange by referencing predefined variables in the PLC code. Such an approach

reducing redundancy and ensuring efficient communication.

System Operation

showcases the expected results after Modbus RTU setup and the PLC program's execution based on the developer's logic. As an outcome, the device can be seen communicating with its counterparts via Modbus RTU.

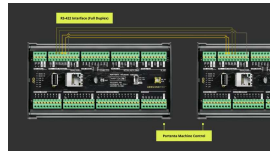
The diagram emphasizes the consistent Modbus RTU configuration across Arduino devices with the PLC IDE. This system's strength is its flexibility. No matter the Modbus RTU specifics, the PLC program can operate seamlessly. Its robust design makes it versatile across diverse settings.

Additionally, the PLC code development is not restricted to one language. The system complies with the IEC61131-3 standard, allowing the option to choose from the languages mentioned in the standard. This helps balance between ease of use and precise code development.

With this broader perspective, we are ready to delve into the details.

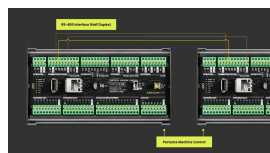
Instructions

'Half-Duplex,' while RS-422 will be configured for 'Full-Duplex' mode. The following image shows the connection diagram for working with two Portenta Machine Control devices.



RS-422 interface (Full-Duplex mode)
between Portenta
Machine Control
devices for Modbus
RTU

The previous illustration resembles a setup of two Portenta Machine Control devices communicating over the RS-422 interface in full-duplex mode. The accompanying image shows an alternative mode of establishing two Portenta Machine Control devices over the RS-485 interface in half-duplex mode.

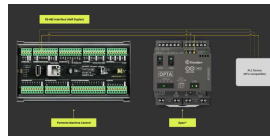


RS-485 interface (Half-Duplex mode)
between Portenta
Machine Control
devices for Modbus
RTU

Both configurations can be used to establish a Modbus RTU-

integrating additional protocol-compatible devices such as Portenta Machine Control or Opta™ (RS-485 interface, Half-Duplex compatible).

The diagram below can help you understand how to connect a Portenta Machine Control and an Opta™ device via RS-485:



RS-485 interface
between Portenta
Machine Control and
Opta™ for Modbus
RTU

The Portenta Machine Control has integrated RS-485 120 Ω differential cable termination.

Because Opta™ has no internal termination resistors, it must be installed per the Modbus protocol specification, adding termination resistors. Also, if you experience inconsistent data transmission using Opta™ with other Modbus RTU-compatible devices, please try again by inverting the A(-) and B(+) lines.



Workspace Pre-Configuration

Before properly enabling and
the Modbus RTU on Portenta

considerations must be considered before using PLC IDE for appropriate Modbus RTU operation. The following subsections will briefly explain such aspects.

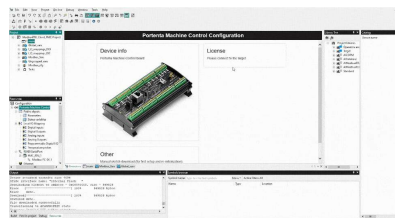


Check out [this tutorial](#) to familiarize yourself with the Arduino PLC IDE environment.

Portenta Machine Control Basic Configuration

The Modbus RTU communication for the Portenta Machine Control does not require special pre-configuration. You will only have to make a manual sketch download with the desired protocol role, its properties, and its onboard elements to be used. These onboard elements can be programmable digital I/Os and dedicated digital outputs.

The Portenta Machine Control can be initialized further in the process if additional configuration changes are required.



Modbus RTU Client and Server Mode

The PLC IDE software allows the

set as either a Modbus RTU Client, Server, or none of the previous modes. It also provides a *Port type* option for the RS-485 (Half-duplex mode) or RS-422 (Full-duplex mode) interface. The option is configurable under `RS485 SerialPort` listed within the `Resources` tab. The provided instructions are set to use the RS-485 interface, Half-Duplex mode.

The Portenta Machine Control set as a Modbus RTU Client will provide 'Baud Rate' and 'Serial Mode' settings. The baud rate is available from 600 b/s to 115200 b/s. The serial mode offers a set of options conformed with the following elements:

Parity: No Parity, Even Parity, Odd Parity

Data Bits: 8 Data Bits
(Only option for Data Bits)

Stop Bits: 1 - 2 Stop Bits

Alternatively, the Portenta Machine Control set as a Modbus RTU Server requires an additional configuration called `Slave Settings`. It will ask you to define the Modbus address with a range between `1 .. 247`.

If you want to disable Modbus RTU for the Portenta Machine Control, you can select the option `none`. This will hide every Modbus RTU configuration parameter, turning off the protocol mode for the Portenta Machine Control.

General Modbus Node Configuration

The General Modbus Node allows the addition of information regarding the devices compatible with the Modbus messaging service.



Arduino PLC IDE -
General Modbus
Node Configuration

It will require you to fill in basic information under the **General** tab and parameters to manage under the **Parametrization** tab. The basic information consists of:

- Name of the device to be communicated under Modbus protocol
- Modbus address (1..247)
- Minimum polling time [ms]
- Address type
- Swap words mode

This information will help identify and correctly communicate with the target device.

PLC IDE Modbus Custom Editor

The Modbus Custom Editor allows you to define a Modbus node with predefined parameters and variables. You can later add it using the 'Add' option under

RS485 SerialPort.

To open the Modbus Custom Editor window, go to

Tools -> Run Modbus Custom Editor

on the PLC IDE.



Arduino PLC IDE -
Modbus Custom
Editor Configuration

It is a helpful feature that saves frequently deployed device configurations compatible with the Modbus protocol.

Project Overview

The example project will test the Modbus RTU connection between two Portenta Machine Control devices after you have prepared the necessary prerequisites and tools.

To create a live handshake verification procedure between two Portenta Machine Control devices, the example project will slightly modify its default example code using the counter ('cnt') variable and broadcast the counter data.

Using the counter data from the 'Modbus RTU Server Portenta Machine Control', the 'Modbus RTU Client Portenta Machine Control' manages the programmable digital I/Os and digital outputs. Each Portenta

simple task using the abovementioned elements. You will learn to configure the Modbus RTU role for each Portenta Machine Control device using the sections dedicated to each role.

If you want to test the entire example project immediately, [you can access it here](#). Every setting and component is ready to be assembled and uploaded to the corresponding Portenta Machine Control.

The following sections will demonstrate how to set up each Portenta Machine Control according to its function in a Modbus RTU connection.

Modbus RTU Server Portenta Machine Control

To configure the Portenta Machine Control as a Modbus RTU Server, navigate to the `RS485 SerialPort` tab in the `Resources` panel within the PLC IDE. It will open a `Modbus Configuration` window where you should choose the `Modbus RTU Slave` setting. For the tutorial example, we will employ the following properties for the server Portenta Machine Control:

Port type: `RS485`

Baud Rate: 19200 b/s

Serial Mode: N,8,1 (No parity, 8 data bits, 1 stop bit)

Slave settings (Modbus



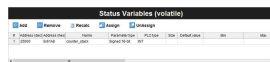
Arduino PLC IDE -
Portenta Machine
Control Server
Modbus Configuration

Alternative values can be used
per requirements if needed.

The subsequent image displays
the

Status variables (volatile)

window. This window will define
the counter_stack variable,
specifying its access address and
data type for Modbus RTU
transmission.



Arduino PLC IDE -
Portenta Machine
Control Server Status
Variable

The counter_stack status
variable uses the following
parameters:

Address: 25000 (dec) /
0x61A8 (hex)

Name: cnt

PLC type: INT

After finishing the settings, go to

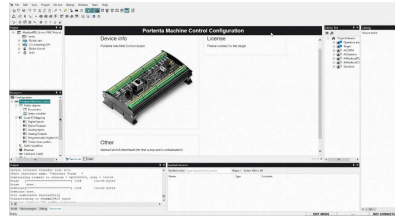
Resources -> Portenta
Machine Control

and select the corresponding
port, beginning the

Manual sketch download

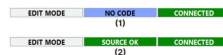
process. Navigate to

and activate Modbus RTU, ensuring that the elevated USB port number designated for the Portenta Machine Control is selected.



Proceed with

On-[line](#) -> [Connect](#) to set up a connection with the Portenta Machine Control server. If everything is set up correctly, we will see a message in the bottom right of the PLC IDE software that says it is connected.



Arduino PLC IDE -
Device Connection
Status

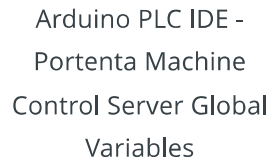
Symbols [\(1\)](#) and [\(2\)](#) show different connection statuses: Portenta Machine Control connected without any PLC code and the Portenta Machine Control connected with an available PLC code, respectively.

To get the main PLC code onto the Portenta Machine Control, go to the [main](#) tab in the [Project](#) section. Then, you can enter the following code:

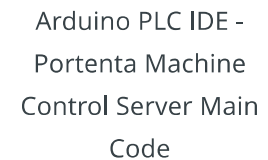



```
1 counter_buffer := cou
2
3 IF counter_buffer >=
4     IF cnt < 255 THEN
5         cnt := cnt +
6     ELSE
7         cnt := 0;
8     END_IF;
9     counter_buffer :=
10    counter_stack :=
11 END_IF;
12
13 // Translate count to
14 DO_0 := cnt AND 1;
15 DO_1 := SHR(cnt,1) AN
16 DO_2 := SHR(cnt,2) AN
17 DO_3 := SHR(cnt,3) AN
18 DO_4 := SHR(cnt,4) AN
19 DO_5 := SHR(cnt,5) AN
20 DO_6 := SHR(cnt,6) AN
21 DO_7 := SHR(cnt,7) AN
22
23 IF counter_stack > 50
24     counter_stack :=
25 END_IF;
```

The main function of the Portenta Machine Control server is to use a binary counter, which is programmed with digital outputs parameterized by a sub-counter variable named `cnt` — this `cnt` variable cycles within an 8-bit map value. The pace of the counter is adjusted using the `counter_buffer` and `delay_buffer`, which serve as customizable timing factors. For the client Portenta Machine Control, the shared Modbus counter variable will be used and is defined as `counter_stack`.



To upload the main PLC code to the Portenta Machine Control, select **Download PLC code** or hit **F7**. Once everything is in place, a successful upload will look like the image below.



Upon completing these steps, you will have effectively set up a Portenta Machine Control device as a Modbus RTU Server. The following section will guide you on configuring another Portenta

Machine Control as a Modbus RTU Client.

Modbus RTU Client Portenta Machine Control

To configure the Portenta Machine Control as a Modbus RTU Client, navigate to the `RS485 SerialPort` tab in the `Resources` section of the PLC IDE. This action will reveal the `Modbus Configuration` panel, where you should choose the `Modbus RTU Master` setting. For the tutorial example, set the client Portenta Machine Control with these specifications:

Port type: `RS485`

Baud Rate: 19200 b/s

Serial Mode: N,8,1 (No parity, 8 data bits, 1 stop bit)



Arduino PLC IDE -
Portenta Machine
Control Client Modbus
Configuration

Alternative values can be used per requirements if needed.

To establish communication with the pre-configured Modbus RTU Server Portenta Machine Control, add a Modbus node by right-clicking the `RS485 SerialPort` tab within the `Resources` section. Once

'Generic Modbus' node. For this example, configure the node with the following parameters:

Name: PMC_RTU_1

Modbus address: 10

Minimum polling time: 1 ms

Address type: Modbus

Swap words mode: Little-endian compliant (No words swapping)

Use the configuration model applied to the Modbus RTU Server Portenta Machine Control for these settings. The vital detail to consider is the Modbus address. Ensure this address corresponds with the server Portenta Machine Control or any other compatible device in case more nodes are added. The setup should resemble the image below:



Arduino PLC IDE -
Portenta Machine
Control Client Node

After setting up the Modbus node for the client Portenta Machine Control, we need to specify the Modbus function responsible for retrieving the counter (`cnt`) data from the server Portenta Machine Control. Right-click on `PMC_RTU_1` or any other name you set with to see the 'Add' option, which will bring forth a device catalog window



Arduino PLC IDE -
Modbus Functions

To retrieve the counter data from the server Portenta Machine Control, select the 'Modbus FC-04 (Read Input Registers)' function. Configure the 'General' tab with the following parameters to ensure correct data access:

Start address: 25000

Polling time: 0 ms
(Continuous Read)

Timeout: 1000 ms



Arduino PLC IDE -
Modbus Functions

Next, you must assign a variable to hold the counter data captured from the server Portenta Machine Control. Go to the Modbus function configuration menu's **Input Reg.** tab. Create a variable named **counter_rec** to store the data sent via the protocol.

The following image shows a visual representation of the anticipated configuration:

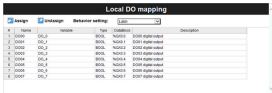


Arduino PLC IDE -

Function of the Node
(Input Reg.)

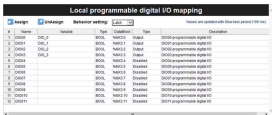
In this tutorial's demonstration, the client Portenta Machine Control is configured to use digital programmable I/Os and digital outputs.

The image below shows how digital output configuration should look within the PLC IDE interface:



Arduino PLC IDE -
Portenta Machine
Control Client Digital
Outputs Table

The digital programmable I/Os also needs labels to reference later in the main PLC code. A table displaying the variable names designated for digital programmable I/Os can be seen below:



Arduino PLC IDE -
Portenta Machine
Control Client Digital
Programmable I/O
Table

The main program below will fetch counter data, control programmable digital I/Os, and manage corresponding digital outputs. A successful Modbus

RTU communication will process previous tasks accordingly.

```

1  counter := counter_r
2
3  IF counter >= 10 THE
4      DIO_0 := 1;
5  END_IF;
6
7  IF counter >= 20 THE
8      DIO_1 := 1;
9  END_IF;
10
11 IF counter >= 30 THE
12     DIO_2 := 1;
13 END_IF;
14
15 IF counter >= 40 THE
16     DIO_3 := 1;
17 END_IF;
18
19 IF counter >= 50 THE
20     DIO_0 := 0;
21     DIO_1 := 0;
22     DIO_2 := 0;
23     DIO_3 := 0;
24     server_opCycle :=
25 END_IF;
26
27 // Translate count t
28 DO_0 := server_opCyc

```

The `counter` variable is a global reference for the client Portenta Machine Control. On the other hand, `counter_rec` is specifically used for Modbus transactions, storing data received from the server Portenta Machine Control, which is related to the `counter_stack` data. This variable was set up during the 'Read Input Registers' Modbus function configuration.

The client Portenta Machine

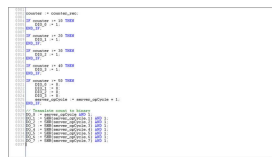
complete digital outputs serving as an operation indicator. Each time the `counter` marks a new tenth value, it will activate the digital programmable output corresponding to the first digit of that counter value. For instance, when the `counter` reaches `10`, digital programmable output #1 will be activated.

Once the `counter` hits `50` and resets, one operation cycle is completed. This process will continuously loop, using the digital outputs as a process cycle counter displayed as a binary counter. The cyclic counter value is stored in the `server_opCycle`.



Arduino PLC IDE -
Portenta Machine
Control Client Global
Variables

With this, we will compile and upload the main PLC code. The interface for the client Portenta Machine Control should mirror the image shown below:



Arduino PLC IDE -
Portenta Machine
Control Client Main
Code

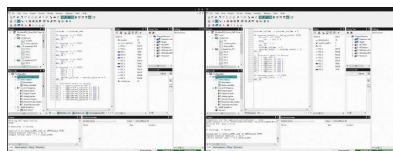
The Portenta Machine Control is now ready as a Modbus RTU Client.

Testing Modbus RTU Communication Between Portenta Machine Control Devices (PLC IDE)

You can access the complete example project [here](#). You can download the compressed file, extract it, and use the pre-configured example project for your Portenta Machine Control devices.

Set both Portenta Machine Control devices to run with the corresponding main PLC code, with the hardware setup explained in [this section](#).

The following short clip briefly shows the expected behavior of the example project.



The server Portenta Machine Control (on the right side) will:

- Execute a binary counter with digital outputs as visual indicators within an 8-bit boundary.

- Increase the shared Modbus counter variable each time the 8-bit binary counter completes a cycle.

- Adjust the binary counter's speed by modifying buffer

The client Portenta Machine Control (on the left side) will:

- Acquire server counter data through the Modbus protocol.

- Interpret and activate the corresponding programmable digital I/Os.

- Use the complete cycle of the programmable digital I/Os to represent the total number of operation cycles, visualized using digital outputs as a binary counter.

Conclusion

In this tutorial, you have learned to configure the workspace environment to work with Modbus RTU using Arduino PLC IDE and verified that a Portenta Machine Control has been correctly set up. Modbus RTU communication effectively uses an example project that controls the Portenta Machine Control device's onboard features, such as programmable digital I/Os and dedicated digital outputs based on customized examples.

Next Steps

Now that you have learned to implement the Modbus RTU between Portenta Machine Control devices using Arduino PLC IDE try adding additional Modbus RTU-compatible devices and creating a Modbus RTU communication network.

Further, the possibilities can be explored by combining the Portenta Machine Control device's onboard features with the Modbus RTU communication network and deploying it to enhance industrial management system solutions.

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

